

Multi-Robot Search and Rescue

Final Report

Team F

Charvi Gupta | Indraneel Patil | Jaekyung Song
Narendar Sriram | Rachel Zheng

Sponsors: Prof. Katia Sycara | Jaskaran Grover

Date: December 16th, 2022





Master of Science
Robotic Systems Development

1. Abstract

Search and rescue is a complex and time critical task wherein the victims must be located, mapped and rescued in a finite period of time. Often these scenarios are dangerous or difficult to traverse for human first responders so robots have been used to fill this need. Target areas can also be really large wherein single robots are rendered useless considering the time constraints. Hence this project aims to deploy a fleet of robots to perform the search aspect of 'search and rescue' while meeting our time sensitive criteria, map and report locations of the victims to first responders so that first responders can do the rescue. The proposed system of RoboSAR works in tandem with human first responders so includes a graphic user interface and similar features. The system also implements efficient cooperation and coordination amongst the fleet of robots to meet our time constraints using a centralized server architecture. Using a complex server architecture allows the system to perform this complex task of search and rescue with very simple robots. The system also includes a very scalable fleet size based on the area of the search and the structure of the environment. This also makes the system very robust against robot failures and other communication losses.

This report describes the requirements, design and implementation of the RoboSAR system and also includes the project management while prototyping the final system.

1. Abstract **2**

2. Project description **5**

3. Use case **5**

4. System-level requirements **6**
4.1 Mandatory Requirements 6
4.1.1 Mandatory Performance Requirements 6
4.1.2 Non-Functional Requirements 7
4.2 Desirable Requirements 7
4.2.1 Desirable Performance Requirements 7
4.2.2 Desirable Non-Functional Requirements 7

5. Functional architecture **8**
5.1 Server Functional Architecture 8
5.2 Agent Functional Architecture 10

6. System-level trade studies **10**
6.1 Centralized vs Decentralized Multi-Robot System 10
6.2 Planning Based vs Control Based Multi Robot Navigation 11
6.3 Multiple vs Single Pose-Graph 12
6.4. Choice of multi-robot simulator 13

7. Cyberphysical architecture **13**
7.1 Server Cyberphysical Architecture 13
7.2 Agent Cyberphysical Architecture 15

8. System description and evaluation **16**
8.1 Subsystem Descriptions 16
8.1.1 Fleet Management System 16
8.1.2 Multi-Robot SLAM 17
8.1.3 Perception 20
8.1.4 Navigation 20
8.1.5 Task Generation 22
8.1.6 Task Allocation 23
8.1.7 GUI 26
8.1.8 Simulation 26
8.2 Modeling, Analysis, and Testing 27
8.3 FVD Performance Evaluation 27
8.4 Strong and Weak Points 28
8.4.1 Strengths 28

8.4.2 Weaknesses 28

9. Project management **29**

9.1 Schedule 29

9.2 Budget 30

9.2.1 Parts List 30

9.2.2 Budgeting Process 30

9.3 Risk Management 30

10. Conclusions **30**

10.1 Lessons Learned 31

10.2 Future Work 31

11. References **33**

2. Project description

Traditional search and rescue is cost, time and labor intensive, and is often fraught with dangers. Rescuers frequently expose themselves to hazardous situations with little information to go on, and time is rarely on their side. To reduce cost and risk while improving search time, we propose integrating robots into the process.

Rather than sending in a team of humans to search a building from the beginning, our project will send in a team of robots, called agents. These agents will then search the rooms and hallways of the building, reporting positions of found victims and overall system status to the end user. Once every room has been searched, the system will report that search has been completed, and the agents will return to the end user.

By sending in robots first, we can quickly locate victims without putting the rescuers at risk. Armed with this information, the human rescue team can then rapidly plan and rescue people while minimizing risk to themselves.

The primary focus of the project is the multi-agent search algorithm: how to most efficiently search a known environment with multiple agents, how to best allocate tasks to each agent, and how to implement the system to minimize search time. RoboSAR gets its name from robotic search and rescue, specifically using a multi-robot fleet.

3. Use case

A school has received a bomb threat. The faculty and students are quickly evacuated. However, after doing a headcount, the teachers realize there are some students missing. The police are sent to rescue the remaining students in the school. However, it is very dangerous for the police to go into the building for extended periods of time, and there is a persistent risk of an explosion. Instead of risking more lives, a swarm of robots is deployed to search the school for trapped victims (fig 1).

Based on a prior floor plan in the form of an occupancy grid like one shown in the image below, the robots search the entire school for students. The swarm of robots all start in one location outside of the main hall doors. The central server, controlled by law enforcement, assigns each robot an area in the school to search. When the robots are navigating the school hallways towards their assigned area, they move in swarm formation so that they do not collide with one another.

The robots branch off from the swarm to search each individual room. When they detect a student, they send the student's location to the police for rescue. A police officer can then head to the student's location and evacuate them to safety. Once a robot is done searching a room, it receives the next room to search, and begins to navigate there. If a robot is lost due to connection error, navigation hazards, or other causes, the central server reallocates the search tasks to the remaining robots. The robots finish searching after visiting every room in the school. Now, the entire school has been quickly and efficiently searched while minimizing risk to both students as well as law enforcement officers.

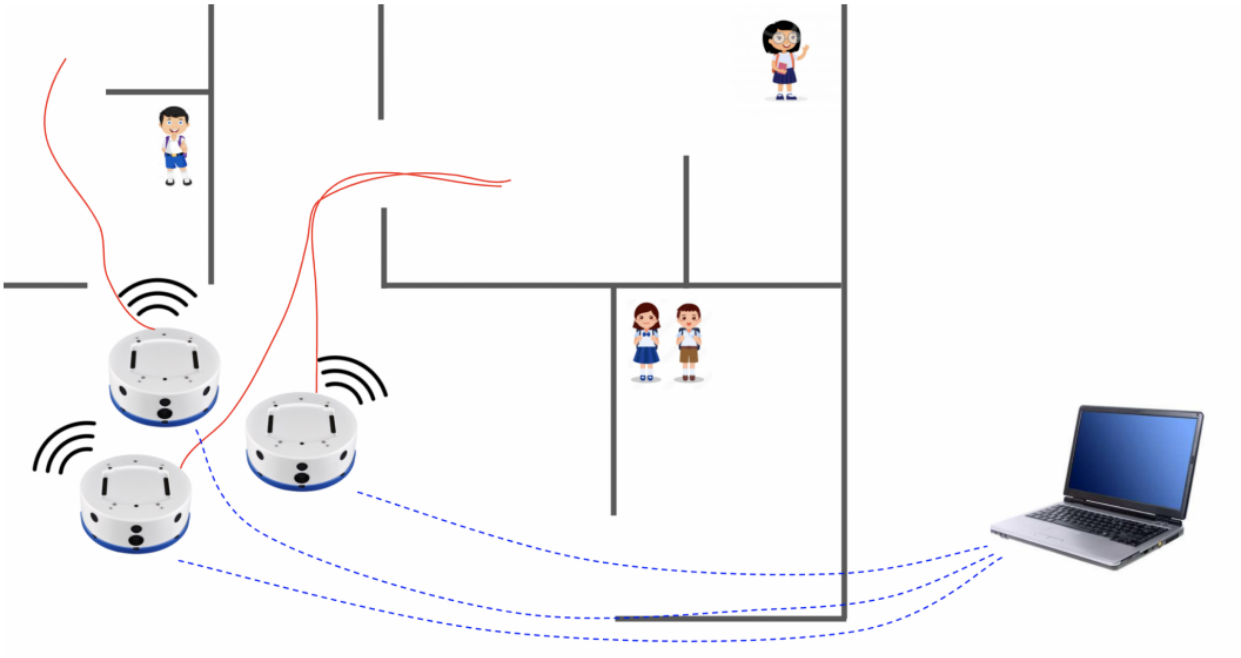


Fig 1. Multi-robot search and rescue

4. System-level requirements

4.1 Mandatory Requirements

Table 1. Mandatory Requirements

M.F.0.	System shall identify and localize victims in the environment using fiducial markers within desired rescue time.
M.F.1.	System shall localize robots with a known map of the environment.
M.F.2.	System shall dynamically allocate tasks to agents in the fleet for optimizing time required for the search.
M.F.3.	System shall ensure simultaneous navigation of agents which have been allocated a task.
M.F.4.	System shall adapt to changes in the fleet size like addition of agents by the users or loss of robots during operation.

4.1.1 Mandatory Performance Requirements

Table 2. Performance Requirements

M.P.0.	System will detect 75% of victims in the environment.
M.P.1.	System will localize victim positions within 5m.

M.P.2.	System will finish searching the environment 20% faster than the naive algorithm for a typical indoor environment.
M.P.3.	System will have downtime of less than 5s for busy agents.
M.P.4.	System will update agent status for the user at 1Hz frequency.
M.P.5.	System will handle at least 3 agents in the fleet.
M.P.6.	System will search 70m^2 within 3 minutes.

4.1.2 Non-Functional Requirements

Table 3. Non-Functional Requirements

M.N.0.	System shall visualize the environment, locations of victims, and robot agents during operation.
M.N.1.	System shall home agents on emergency stops, and after completion of search.
M.N.2.	System shall have a user interface to add/subtract robots during operation.
M.N.3.	System shall be well documented and be a reusable software framework for swarm robotics.

4.2 Desirable Requirements

Table 4. Desirable Requirements

D.F.0.	System shall map and search in an unknown environment simultaneously during operation.
D.F.2.	System shall support flocking/formation control of multiple robots executing similar trajectories.
D.F.3.	System shall account for victims needing simultaneous detection by multiple robots.
D.F.4.	System shall account for victims needing detection by specific types of robots.

4.2.1 Desirable Performance Requirements

Table 5. Performance Requirements

D.P.0.	System will finish searching the environment 20% faster than the naive algorithm in an unknown environment.

4.2.2 Desirable Non-Functional Requirements

Table 6. Non-Functional Requirements

D.N.1.	System shall visualize victims needing multiple or special robots.

D.N.2.	System shall support heterogenous robot search and control.

5. Functional architecture

The system is divided into two parts: the server and the agents. A high level overview of the two parts is shown in figure 2.

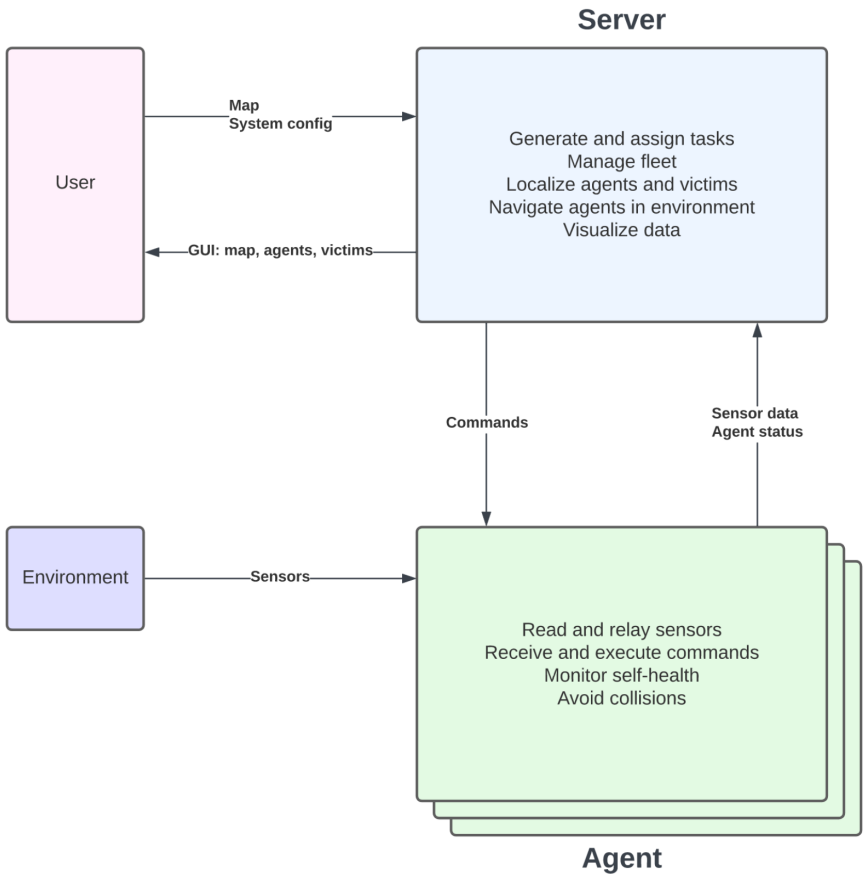


Fig 2. System Functional Architecture

5.1 Server Functional Architecture

Shown in figure 3, the server sends and receives information with the agents, and also provides an interface to the user. To start **exploring** the environment, the user provides an initial map of the building, showing the hallways, rooms, and walls. As the agents explore the environment and discover victims, they will report this information along with their own poses to the server. The server will then visualize the world state for the user through a GUI.

The map is also used for **task generation and allocation**. Using the map, the server will identify the best areas to explore. At the same time, the server will track the number and status of agents with a **fleet manager**, and create a list of available agents. Task allocation will then use this list and the identified points-of-interest to assign tasks to the agents.

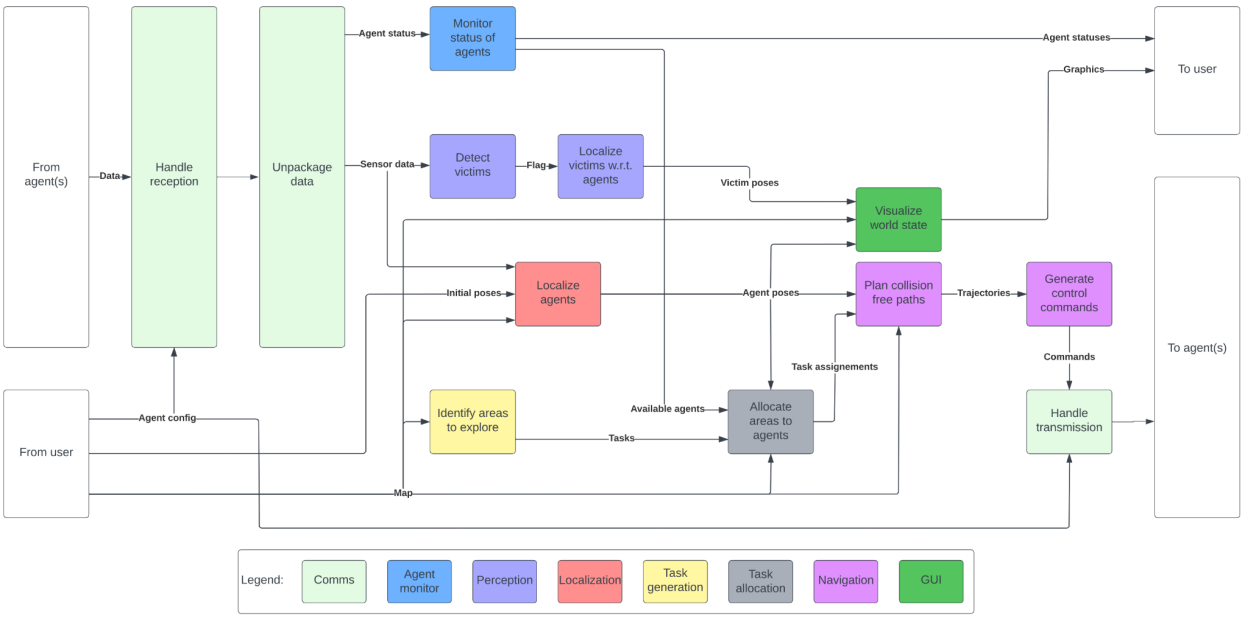


Fig 3. Server Functional Architecture

Last is **navigation**: collision-free paths are generated in the map for the agents to follow. Additionally, the server must control the execution of these paths to ensure the agents are moving as needed, which is done by comparing the desired position in time with the current agent position. This comparison is used to compute linear and angular velocity commands, which are sent to the agents.

5.2 Agent Functional Architecture

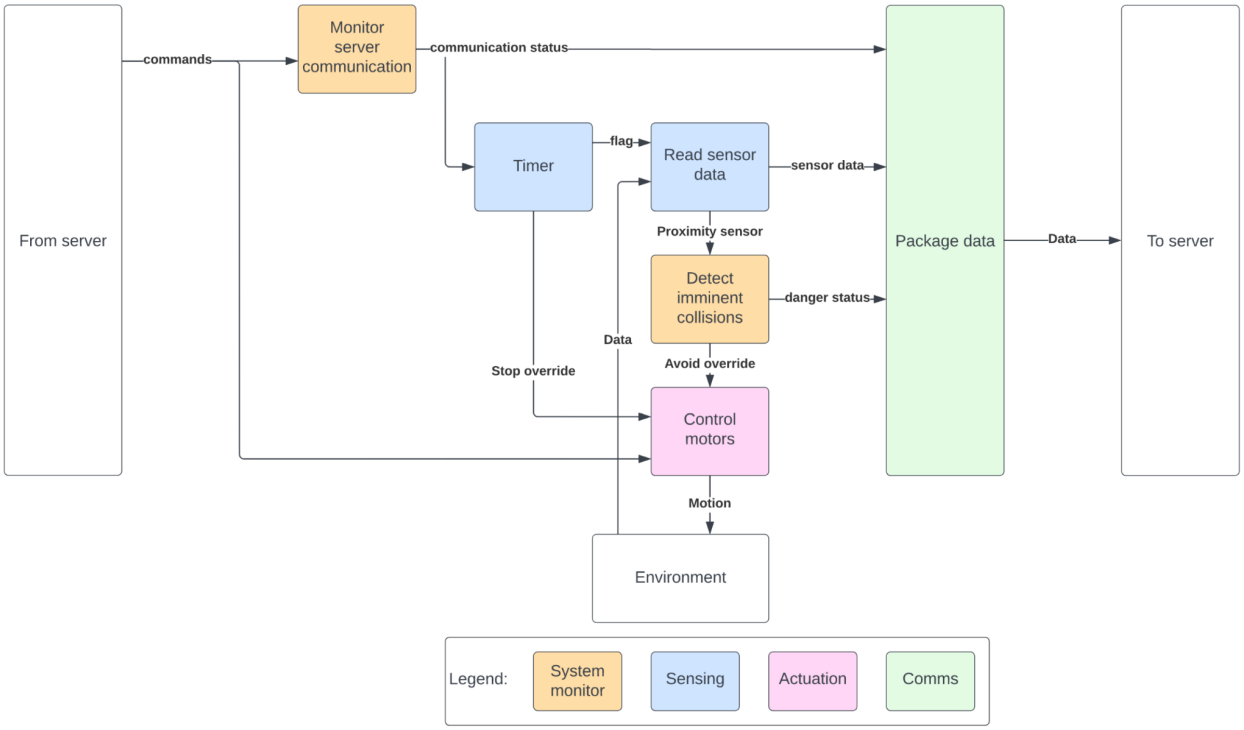


Fig 4. Agent Functional Architecture

The agent explores the environment, and reports sensor data to the server, as shown in figure 4. First, the agent receives linear and angular velocity commands from the server. This is used to compute **movement** by comparing its current velocities with the server-calculated velocities. The agent's motors are actuated to minimize this difference, moving the robot, thus achieving exploration.

As the agent moves, it reads its sensors to perform **perception**. The data from the sensors are analyzed to victims as well as collision avoidance. This information is transmitted to the server to update the world-state.

The agent is always **monitoring** its own status, both in terms of health and imminent collisions. The agent performs self testing as well as checks the outputs of its sensors to detect failures. If any occur, the issue is reported to the server so the server can decide what to do with the malfunctioning agent.

6. System-level trade studies

6.1 Centralized vs Decentralized Multi-Robot System

This trade study (table 1) highlights the main design choice that we had to make in the pre functional architecture phase. The choice of whether the multi-robot team will have a centralized control (multi-agent system) or a decentralized control (swarm intelligence). Both

these systems have their own benefits but we have used our system functional requirements as a lighthouse to decide what's truly important to our search and rescue scenario in the evaluation criteria. We have also added a column which indicates the mapping of the criteria to our functional requirements. Amongst all the criteria, optimal task allocation, robot localisation complexity and navigation complexity are the most important to us because the efficiency of search and rescue and the feasibility of our system depends on it. Followed by other criteria which should be considered but not as important, such as formation control, scalability of fleet and search area size, robustness to failures.

Table 1. Multi-Robot Control Trade Study

Criteria	Source Functional requirement	Weight Factor	Value (1 to 5)	
			Decentralized	Centralized
Optimal Task Allocation	M.F.2	30	1	5
Flocking/Formation control	D.F.2	5	5	3
Robot localisation complexity	M.F.1	25	1	5
Size of robot fleet/search area	M.F.4	5	5	1
Kin Recognition/Navigation complexity	M.F.3	25	1	5
Robustness to Robot Failures	M.F.4	5	5	1
Operator Graphical User Interface	N.F.0	5	3	5
Weighted Average		100	1.7	4.5

6.2 Planning Based vs Control Based Multi Robot Navigation

For a centralized system there are two main ways of doing multi-robot navigation, a planning based approach and a control based approach. In a planning based approach we implement a centralized planner and a decentralized controller whereas in a control based approach we have a decentralized planner but a centralized controller.

We did not make this trade study (table 2) to drive this decision since we first implemented the planning based approach and then the control based approach when we found out that the former method was unsuitable for the project. We only do this trade study as a retrospective and we find that this trade study would have significantly benefited us in making the right decision on this. All our requirements as shown in the table champion the control based approach. We found that the planning based approach makes stuff like navigating robots with asynchronous task completion and scaling the fleet to more than 3 robots quite difficult. Formation control isn't even possible with a planning based approach.

Table 2. Multi-Robot Control Trade Study

Criteria	Source Functional requirement	Weight Factor	Value (1 to 5)	
			Planning Based	Control based
No down time for allocated agents	M.P.3.	20	3	5
Atleast three agents in the fleet	M.P.5	20	2	5
Flocking/Formation control	D.F.2	10	1	5
Fit for indoor environment	M.P.2	25	3	3
Fit for localisation accuracy	M.F.1	25	5	3
Weighted Average		100	3.1	4.5

6.3 Multiple vs Single Pose-Graph

In order to operate in an unknown environment, we must construct a map during operation. Since our system has multiple robots, there are two primary ways of going about this. First, we could have each robot generate a map based solely on what it knows, and then stitch the individual maps together to form a unified map; this approach would be utilizing multiple pose graphs. Alternatively, we could have all the robots combine their knowledge first, then generate a cohesive map right away; this would be utilizing a single pose graph. Both approaches have their advantages and disadvantages, and there was disagreement in the team of how best to go about this, so we did a trade study, as shown in Table 3. We decided to go with the single pose graph due to ease of implementation, ease of generating a unified map, and its ability to generate more accurate maps via superior loop closures.

Table 3. Pose Graph Trade Study

Criteria	Weight Factor	Value (1 to 5)	
		Multiple	Single
Ease of implementation	20	4	5
Unified map generation	20	2	5
Individual map generation	5	5	2
Pose graph initialization	10	3	5
Compute load	15	2	3

Multi agent loop closure detection	15	2	4
Multi agent pose graph optimization	15	1	5
Weighted Average	100	2.5	4.4

6.4. Choice of multi-robot simulator

There are a lot of robotics simulators but all of them are not scalable to multi robot applications since some of them are super detailed and hence become computationally inefficient while simulating multiple robots. On the other hand there are multi robot simulators which are so primitive that they dont model simple obstacle interaction. Through this trade study (table 4) we are trying to find a good trade off between simplicity for the scalability required for our multi robot scenario but also good built in collision checking using a physics engine.

Table 4. Simulator Trade Study

		Gazebo	Morse	Webots	Stage	V-Rep	ARGoS
Criteria	Weight Factor	Value(1 to 5)					
CPU Load	25	1	5	3	5	3	5
Multi-robot collision checking	25	5	5	5	3	5	5
Ease of developing new environments	10	3	5	2	5	3	3
Modeling obstacle interaction	10	5	5	3	1	5	1
Documentation	30	5	3	3	5	3	3
Weighted Average	**100**	**3.8**	**4.4**	**3.4**	**4.1**	**3.7**	**3.8**

7. Cyberphysical architecture

Following the functional architecture, the cyberphysical architecture is also divided into two parts. The server runs on a stationary central computer and the agents are out in the field. There are multiple agents and they have the same cyberphysical architecture as depicted in this section.

.

7.1 Server Cyberphysical Architecture

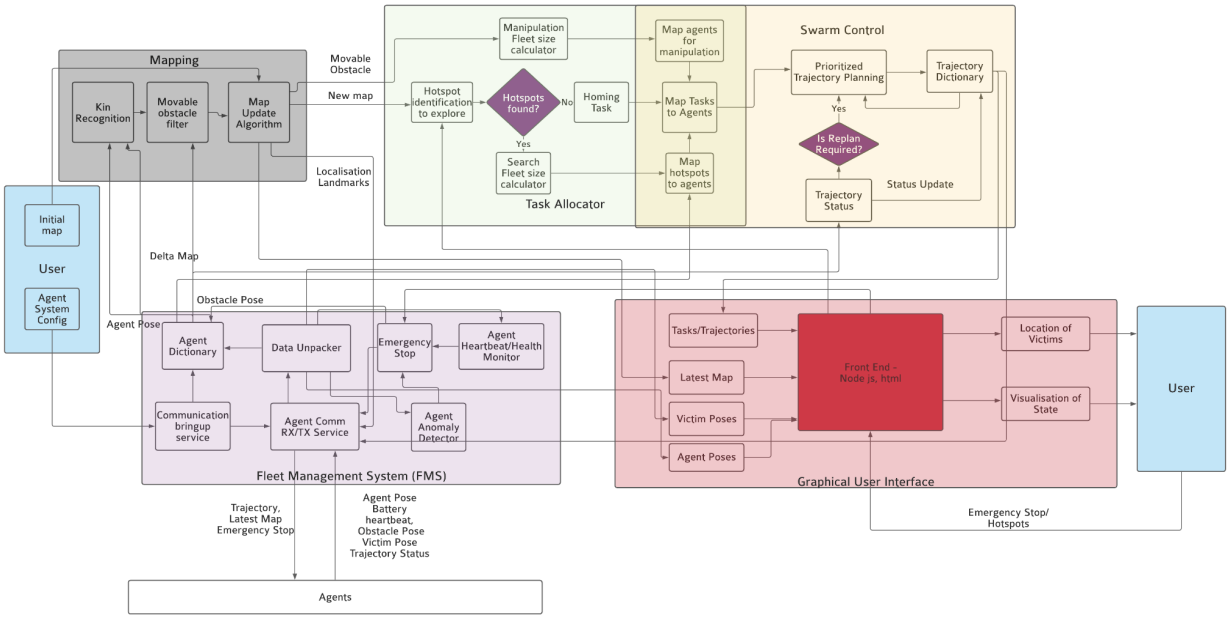


Fig 5. Server Cyberphysical Architecture

The server acts like the brain of the system, interacts with the end user and orchestrates behavior of the agents, as shown in figure 5. It takes the initial map of the environment and a system config file which contains the information of all agents in the system as an input from the user. Then the **Fleet Management system** (FMS) uses the system config file to set up the communication interface with each agent in the fleet. It then passes the communication handler to the RX/TX service and also updates the agent dictionary based on the successful connections. Emergency stop inside the FMS can be triggered from the Anomaly Detector, Health monitor as well as the user. Up to date information about all agents is maintained inside the agent dictionary.

The **Mapping** subsystem is incharge of collecting new sensor data from the FMS and building the map, returning localisation cues to the agent and passing this map to the Task allocator.

Task Allocator subsystem identifies new tasks to undertake, calculates optimal number of agents needed for these tasks and then also maps agents to these tasks. Task allocator performs its function in very close conjunction with the Swarm control subsystem.

Swarm Control subsystem as the name suggests ensures that the swarm of agents complete their tasks simultaneously and efficiently. After planning trajectories for each agent ,it also monitors the execution of these trajectories and replans paths if needed.

All these operations are visualized by the **Graphical User Interface** (GUI) for the user in real time so that the operator is always informed about the search and rescue mission. The GUI also allows the user to trigger emergency stop and insert search hotspot requests into the system.

7.2 Agent Cyberphysical Architecture

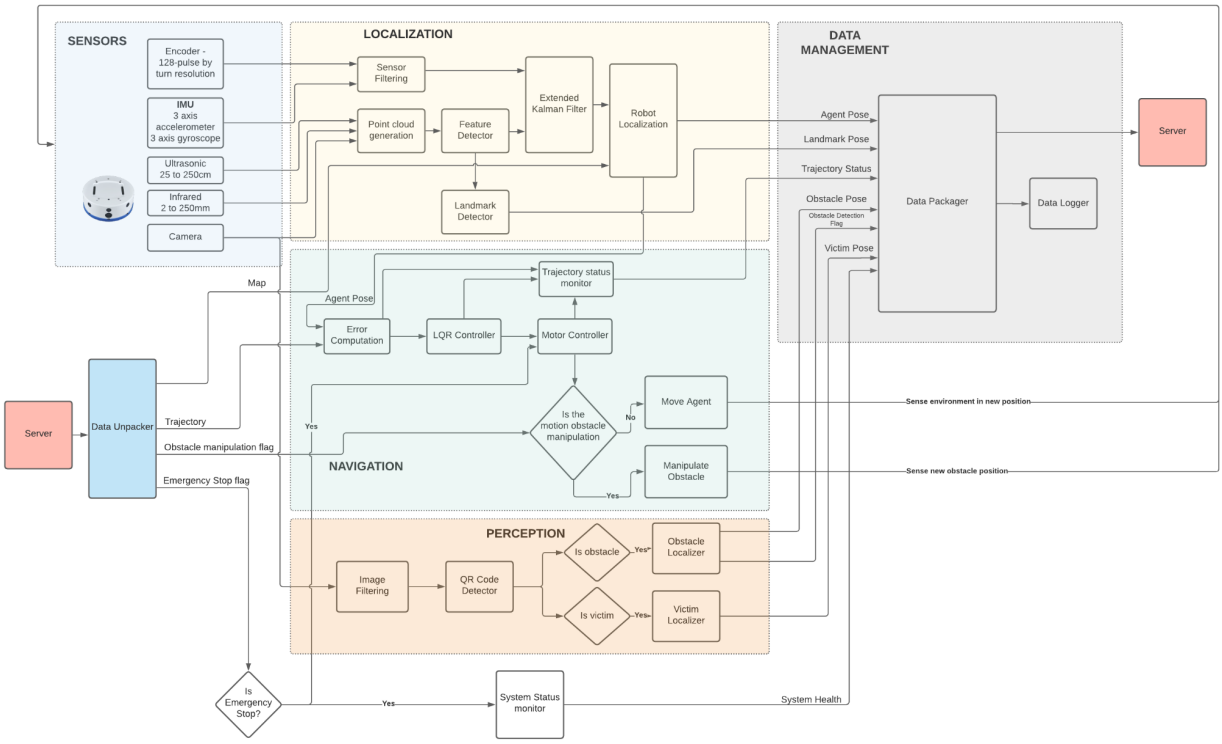


Fig 6. Agent Cyberphysical Architecture

We are using Khepera IV robots for the agents. This platform was primarily designed for testing artificial intelligence algorithms, especially multi-agent systems, so they are well suited for our purpose [3]. The Kheperas come with several **sensors** (as shown in figure 6), which can be used for proprioception (odometry, 3-axis gyroscope and 3-axis accelerometer) as well as measuring the environment (infrared and ultrasonic as range finders, RGB camera).

The sensors are used in the **localization** subsystem to determine agent pose. Proprioception is used for dead-reckoning navigation, which is augmented by environment measurements to form an Extended Kalman Filter (EKF). The output of the EKF is combined with the server map to determine self pose. This subsystem will also detect and localize landmarks, allowing itself as well as other agents to use the found landmark for loop closure.

The **navigation** subsystem uses self pose to track trajectories. The difference between the desired and actual pose will generate an error, which is fed into an LQR controller , which controls the differential steering motor speeds to achieve the desired trajectory. By following the trajectory, the Khepera will explore the environment and manipulate obstacles using a non-prehensile manipulator. A trajectory status monitor will monitor the error, LQR controller, and the motor currents to detect failures, and reports to the server as appropriate.

The Khepera's camera is used to identify and locate both obstacles and victims in the **perception** subsystem. Because obstacles and victims will use QR tags, the Khepera will look for QR tags in its camera stream and read it. Once found, the contents of the tag will identify the

object as a victim or obstacle. The camera feed is then analyzed to determine the victim or obstacle's pose, which is then reported to the server.

The **Data Management** subsystem packages all the information going to the server before logging and transmitting it. Packaging makes the data transfer more efficient, as well as easier to work with on the server side.

8. System description and evaluation

We have 8 total subsystems: Fleet Management System (FMS), Multi-Robot SLAM (MRSLAM), Perception, Navigation, Task Generation, Task Allocation, Graphical User Interface (GUI), and Simulation.

8.1 Subsystem Descriptions

8.1.1 Fleet Management System

The fleet management system (FMS) depicted in figure 7 is the bridge between the server and all the agents in the field. As depicted in the diagram the green functions are the ones which were already implemented in the spring. The red ones are new features from the fall semester. Time clock synchronization was never implemented and that is left as future work.

At a high level, the FMS accepts a config file from the operators which contains information on all the robots in the fleet, then uses that to set up communication channels for transmission and receiving with all the robots in the fleet. It also monitors these connections during operation, decodes sensor data for the ROS network, logs this data in rotating log files and also informs all the other subsystems on any changes in the fleet.

One other strong feature of the simulation was the ability to simulate all the robots using our multi robot simulator discussed later, that meant we could run the same system without hardware with no code changes!

Interesting challenges during testing:

1. We successfully tested this design with 5 robots in the field, as we tried scaling up from 5 robots we had bandwidth issues on our router/local network so we faced problems like packets dropping and very sketchy communication. This needs to be investigated further.
2. Since the chosen communication protocol was UDP, once we entered a natural environment with infrequent line of sight communication we faced all UDP related problems like packets arriving out of sync and packets being dropped midway communication. For the latter, we had a control timeout timer on each robot and for the former we had a filter in place on the server.

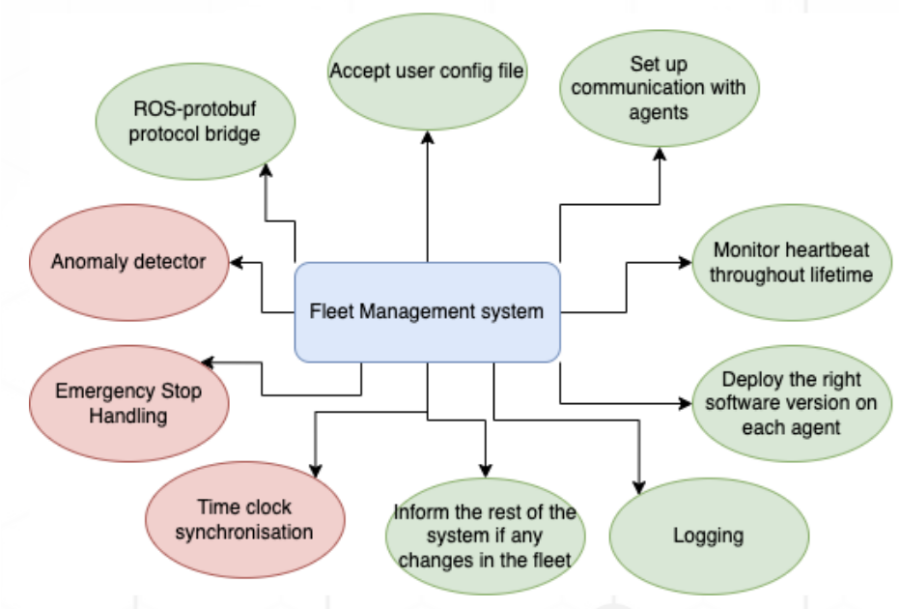


Fig 7: Fleet management system feature map

8.1.2 Multi-Robot SLAM

The Multi-Robot SLAM (MRSLAM) system is responsible for generating a map of the environment, and determining the poses of the agents and found victims within that map. The map and the poses of victims and agents are then output for the rest of the system to utilize. The map and poses are determined using a pose graph SLAM approach:

1. MRSLAM takes in the sensor data from the agents, which are made available by the FMS. The sensor data contains odometry by way of the wheel encoders, 2D laser scan data, and if a victim has been detected, the victim's ID
2. The odometry is used to estimate the pose of the agent, relative to the agent's previous pose. This pose estimate is then refined using laser scan matching, reducing the effects of slip, drift, and noise of the wheel encoders
3. The refined pose estimate, and the laser scan at that pose, are added to a joint pose graph. This pose graph, seen in figure 8, contains information from every agent in the system; in other words, all the agents share a single pose graph. This pose graph has nodes, which represent poses of the agents, and edges, which represent the change in pose, as well as a covariance to represent the uncertainty in this measurement.

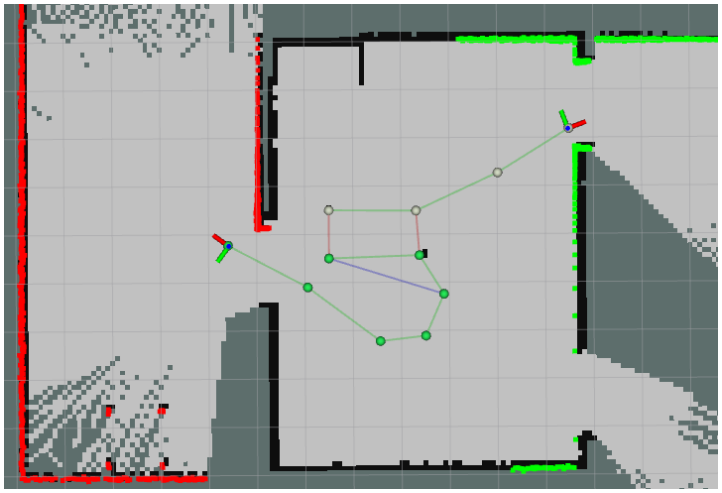


Fig 8. Pose graph visualized, showing where agents currently are and have been

4. Loop closures candidates are found whenever a new node is added to a pose graph. If the new node has a similar pose to a node from the past, then the laser scans from each node are compared. If the two scans are similar, then the loop closure candidate is accepted, and a loop closure occurs. In the pose graph, the loop closure is represented by an edge connecting two non-consecutive nodes.
5. When a loop closure occurs, the pose graph is optimized. A loop closure adds "stress" to the pose graph by forming a closed loop of constraints. It's unlikely that all the constraints can be satisfied at once, so some of these constraints must be violated, leading to "stress." By considering the uncertainty of each constraint, we can weigh how much "stress" we are creating by violating the constraint; this is called the Mahalanobis distance. The optimizer will adjust the past and recent poses to minimize the Mahalanobis distances of the graph overall, giving the most probabilistically likely pose graph, given the sensor measurements. Once optimized, we not only have an improved estimate for all agents at the current time, but we also have improved estimates for all past poses too.

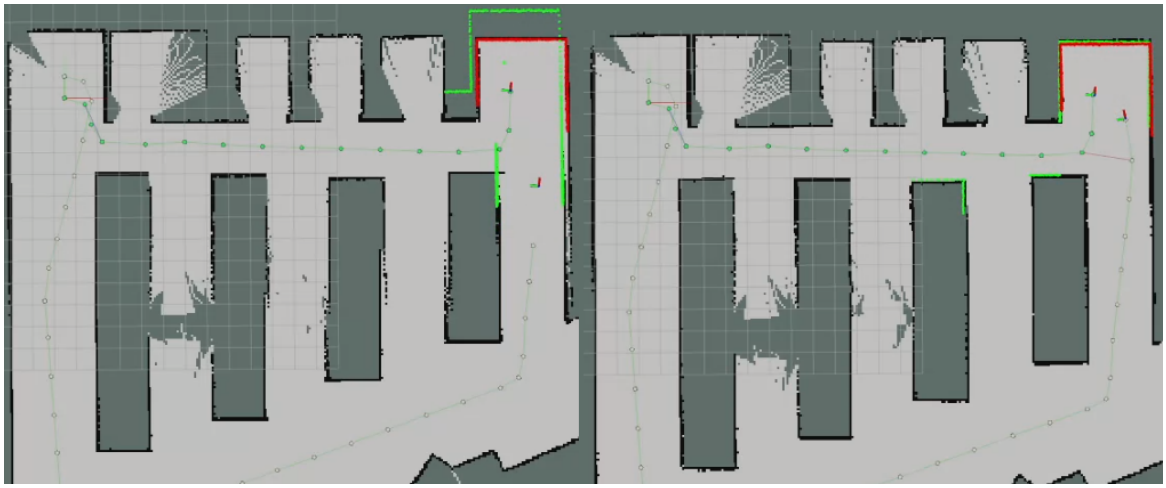


Fig 9. Map, before (left) and after (right) optimization. Note that before optimization, green laser scan does not align with map, the four central rectangles are too short, and the bottom half and top half of the walls of the rectangles do not align properly; optimization fixes these issues.

6. In addition to estimating agent poses, the pose graph is also used to generate a map. Since each node contains a laser scan, we can construct an occupancy grid map. At each node, the node's laser scan is used to perform raytracing; cells where the laser scan passed through are considered free space, and cells where the laser scan stopped are considered occupied cells. Performing raytracing for all nodes, taking into account the agent's pose at each node, will construct a map showing free and occupied space. Because the entire pose graph is used to construct the map, and the pose graph includes information from all agents, the generated map includes information from every agent in the system. When a loop closure occurs and pose estimates become more accurate, seen in figure 9, this causes the map to become more accurate as well.
7. When a victim is detected, the detection is tied to a node, and the pose associated with that node is used to determine the victim's location. This ensures that when a loop closure occurs, and the map changes, the estimate of the victim's location is also updated and remains accurate. This prevents the victim's location from becoming outdated as the map changes over time.



Fig 10. Final map of Tepper locker area, mapped by 5 agents. It was the venue for FVD and FVD Encore.

MRLSLAM proved to be accurate, robust, and computationally light in simulation and in real world testing. It fulfilled the promise of improving accuracy over time as more and more loop closures occurred, ensuring that the map and pose estimates were accurate. This allowed the system to operate well in a variety of environments. The key to robust performance was the tuning of loop closure parameters. While a good loop closure will improve the map, an incorrect loop closure could irreparably damage it, often leading to mission failures. This meant that it was better to miss a good loop closure than to accept bad ones. To accomplish this, tuning of the loop closure candidate selection process, as well as how closely two scans had to match for a loop closure candidate to be accepted, was done repeatedly and over a significant amount of time. The tuned system was tested repeatedly to ensure accuracy and repeatability, leading to MRSLAM being one of the most reliable subsystems in the project (figure 10).

8.1.3 Perception

A key part of RoboSAR is victim detection. The task generation and allocation algorithms work in tandem so that agents can identify and report the maximum number of victims placed in the environment. For our system, victims are represented as AprilTags. Each AprilTag has a unique ID number which represents a victim.

For victim identification, we exploit the use of RGB cameras present on the Khepera robots. It is a MT9V034C12ST from Aptina.

a. The perception template is run on the agents along with template script while initializing the system (as a part of the agent bringup node). The function of this script is to capture a frame and report apriltag id if detected.
b. In order to make sure that the system does not slow down as the frame capture rate is low (as the resolution is maximum 752X480), Inter Process Communication (IPC) has been implemented which means the following:
    i. Template script on Khepera runs at a higher frequency and constantly transmits data to the server.
    ii. Template_perception script runs at a lower frequency then template (1 Hz) and transmits data to the template script should there be any detection.
c. This process makes sure that the system is not bottlenecked by the slow frame capture rate of the Khepera cameras (fig 11).

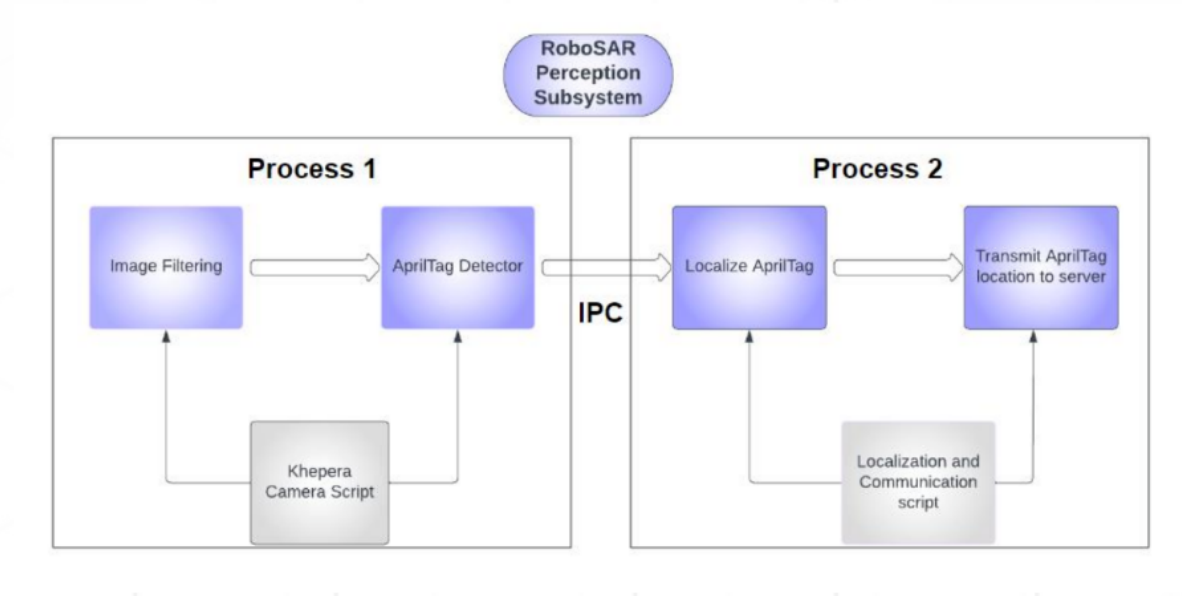


Fig 11. RoboSAR Perception architecture using IPC

d. Search behavior was also implemented as a state machine in each agent. When an agent reaches a coverage task, instead of reporting the goal as "reached" to the server and taking on the next task, the agent exhibits a search behavior by rotating in place and looking out for apriltags. Refer below to figure 12 for the state machine.

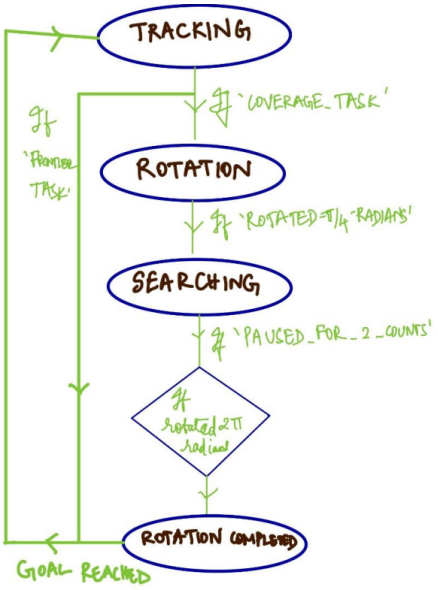


Fig 12: State Machine for search behavior

Thus, by combining the perception template script and search behavior, victims in the environment are detected and reported to the server which in turn are displayed on the GUI.

8.1.4 Navigation

The navigation subsystem depicted in figure 13 takes in the current poses of the robots, and their assigned tasks, as well as a map of the environment, to generate a path from each robot to its task. The controller then computes velocities to track this trajectory. This subsystem ensures that all the agents are able to traverse around each other and navigate from their start positions to their goal locations without colliding with the environment, with one another, crowding, or slowing each other down.

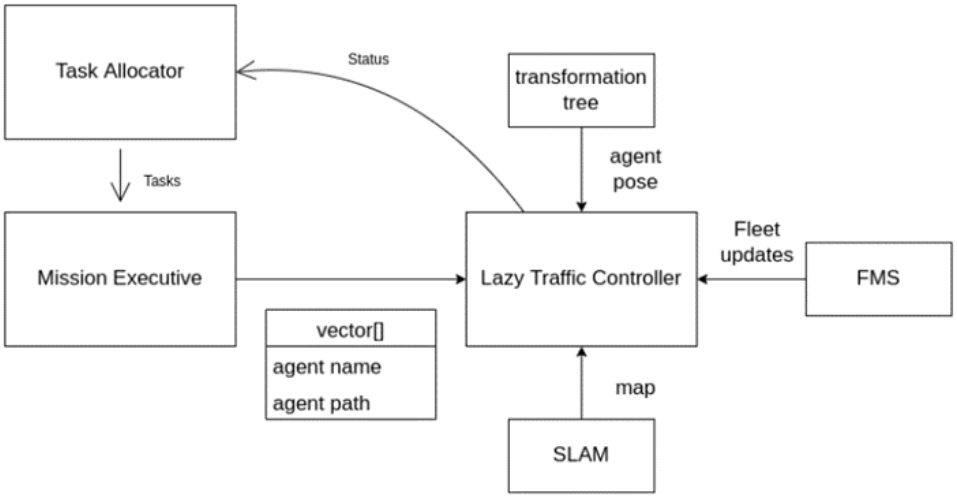


Fig 13. Integration of navigation stack with rest of the system

Decentralized 2D A*

Using a decentralized 2D A* planner, we plan for each agent separately. This planner outputs a collision-free path for each robot only with respect to the occupancy grid map of the environment and does not guarantee robot-robot collision avoidance.

Lazy Traffic Controller

The local collision avoidance between robots is done by a centralized controller. The Lazy Traffic controller takes in the paths from the decentralized planner and is responsible for local robot-robot collision avoidance by calculating collision-free velocities for each robot. It consists mainly of three parts:

i. Velocity Obstacles

For each agent, all other agents act as obstacles in the velocity space and hence invalidate a set of velocities that this agent can execute. Velocity obstacles [8, 9] try to select the best velocity as close to the preferred velocity as possible from the set of valid velocities (figure 14). Preferred velocity is calculated for each agent from the global A* path found by the planner. Staying on this A* path is the preferred thing to do unless otherwise ruled out due to obstacles.

The cost function being used is:

penalty = RVO_SAFETY_FACTOR/min_t_to_collision + dist_to_pref_vel + dist_to_cur_vel

The three terms in the cost function ensure collisions are avoided, a velocity as close to the preferred velocity is chosen every time and large changes in currently chosen velocity direction are also minimized.

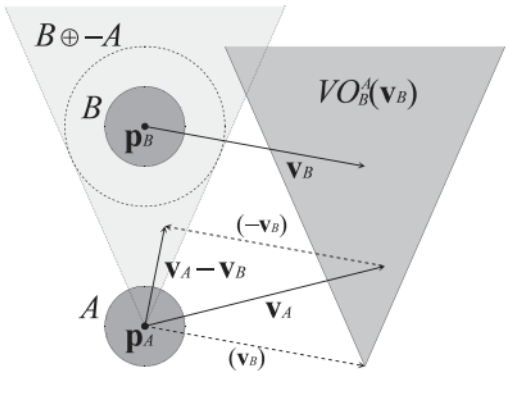


Fig 14. Velocity Obstacles [8]

ii. Static Obstacle Avoidance

LTC's capabilities were limited to agent-agent collision avoidance. There were scenarios where A-Star outputs paths close to the walls. Note that the controller only chooses a velocity close to the preferred A* velocity. It is not forbidden from choosing a non-preferred velocity. It is only penalized. This caused the controller to choose paths which collided with walls in certain cases. Therefore, there was a need to extend the LTC's capabilities to avoid walls and static obstacles. We perform BFS from the agent's location up to a certain radius in its neighborhood on the map published by SLAM (available on the ROS topic /map) and add all the points which are above a collision threshold as static neighbors to the agents. This makes the VO (Velocity Obstacle) treat these points as discs centered on these points with zero velocity and considers them in time to collision and velocity calculations. This prevents the agents from running into walls.

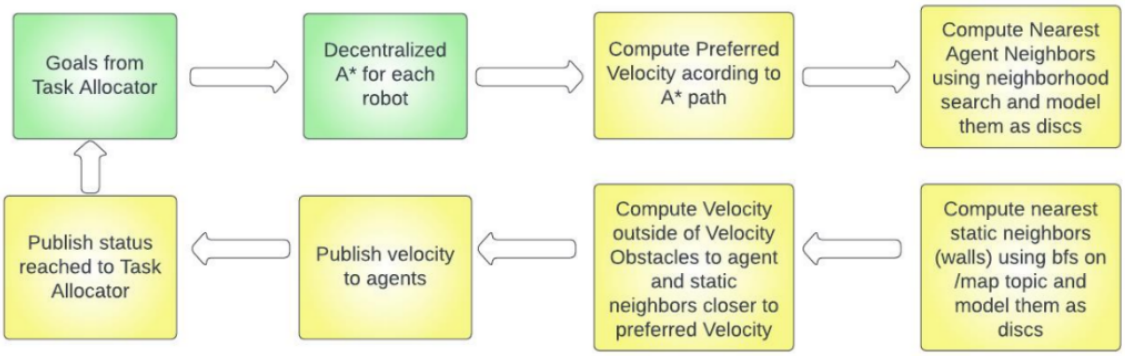


Fig 15. LTC with Agent-Agent and Static obstacles collision avoidance

iii. Inter-Robot Repulsion

On top of velocity obstacles and static obstacles avoidance, nter-robot repulsion (figure 16) is added if there are any agents within the repulsion zone of the ego-agent. This is inversely proportional to the distance between 2 agents and helps to avoid collisions in tight spaces.

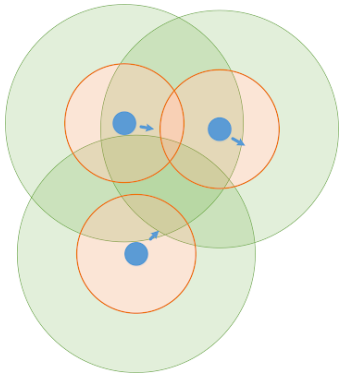


Fig 16. Inter-Robot Repulsion

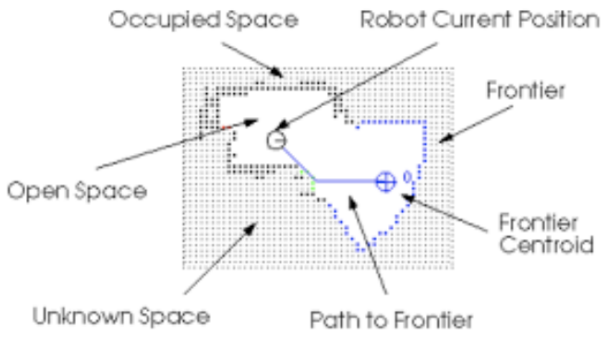
8.1.5 Task Generation

Now that we are performing search in an unknown environment, we:

- Need to be able to generate interest points that help explore the unmapped parts of the environment and exploit the already mapped environment.
    - Since the exploration sensor (Lidar) has a larger range than the exploitation sensor (camera), some mapped out parts of the environments will not have been effectively searched for victims. We will have to generate coverage and frontier tasks.
- Need a unified solution for both coverage planning and frontier exploration [2].
    - We want to be able to generate coverage and frontier tasks with the same system.
- Need to keep a track of visited/unvisited areas in a dynamic map.
    - Since the map will be continuously changing, it will be hard to keep track of which areas have been already visited, and which areas already correspond with newly mapped areas.
- Need a task generation graph which persists through loop closures!
    - The generated tasks should persist throughout the mission and not be deleted/disappear with map updates.

So what are frontier and coverage tasks?

- **Frontier tasks:** tasks that are on the border between mapped and unknown space (figure 17a). If the robots go to these tasks, they should be able to map out and further explore the environment.
- **Coverage tasks:** tasks that are points in the mapped free space that maximize camera sensor coverage. If the robots go to these tasks, they should execute some search behavior so that it can look for potential victims.



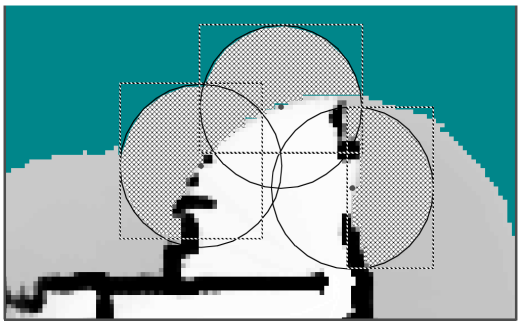


Figure 4: Expected Information Gain
Information gain regions for several representative frontier cells. Circles indicate sensor range. Cross-hatched areas are information gain regions. Dotted lines are the rectangular approximations.

Fig 17. (a) Elements of an occupancy grid [1]. (b) Information gain [2].

Our solution:

- **Pose-Graph Aware Task Generation**
    - Multiple random trees [4] are grown from the pose-graph nodes.
    - Updated with the pose-graph (figure 18).
    - If branches intersect occupied space, they are disabled but can be re-enabled in the future if they no longer are in occupied space.
- **Frontier tasks:**

    - Nodes that connect into unknown space.
    - Filtered based on information gain (figure 17b).
- **Coverage tasks:**
    - Nodes on the trees that maximize sensor coverage.
    - Robot executes search behavior at these tasks.

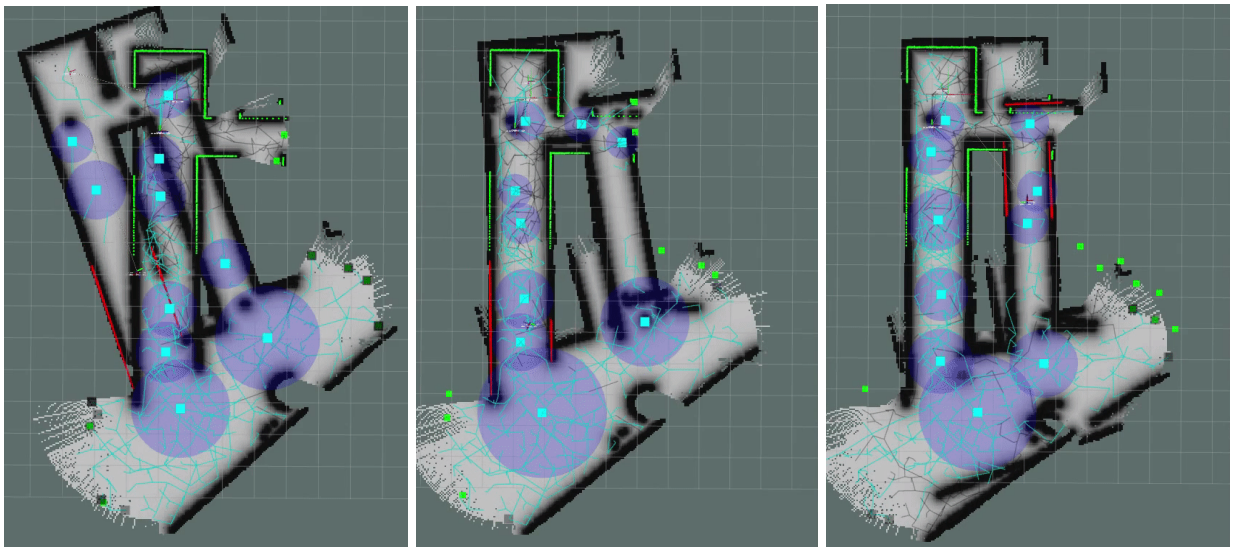


Fig 18. Cyan squares: coverage tasks, Blue circles: coverage task radius, Green squares: frontier tasks, Cyan lines: Pose-Graph Aware Task Generator random trees. (a) Before loop closure: there are two of the same hallway in the built map, so there exists redundant tasks. (b) After loop closure 1: the hallways are now correctly associated, and the redundant tasks are deleted. The random trees (cyan lines) are updated with the pose graph, and minimal pruning (black lines) occurs. (c) After loop closure 2: The random trees are again updated, with no pruning needed!

8.1.6 Task Allocation

The task allocation subsystem efficiently assigns each robot tasks such that all tasks are completed in minimal time.

Now that we have two types of tasks, frontier and coverage tasks, we need a task allocator that can assign each robot to these tasks and make intelligent decisions on which task will reap the most reward each assignment period. We implemented two task allocators: a naive algorithm as a baseline, and a more optimal task allocator, HIGH:

- **Naive Allocation**
    - Greedy based on distance to task and utility
- **HIGH (Hierarchical Information Gain Heuristic) Allocation**
    - Information-theoretic approach: task weight is encoded by its information gain
    - Balances exploration vs exploitation with weights based on explored area

Each task allocator has the following two objectives: the **A* path length** to each task as the distance cost, and an additional **utility** penalty if the task is within line-of-sight distance to an already assigned task. This prevents robots from being assigned tasks that are very close to each other in the same assignment round.

HIGH uses the same costs, but additionally considers exploration of the environment vs exploitation of the already mapped areas for victim search. We are using an information-theoretic approach that weighs the naive costs, information gains, and exploration-exploitation weights [5] to assign each robot to the task with the highest reward. We introduce two new objectives:

- **Information gain** (figure 19)
    - The information gain is [0., 1.]
    - For frontier tasks, information gain is defined as the percent of continuous unknown cells within a radius.
    - For coverage tasks, information gain is defined as the percentage of free cells within a radius that have not yet been covered by the camera sensor.
    - Tasks with high information gain should be prioritized in assignment, and tasks with low information gain should not be assigned.
- **Exploration-exploitation weight**
    - When we first start the mission, we want to rapidly explore the environment. Conversely, when the environment is mostly mapped out, we want to prioritize completing coverage tasks.
    - We use a linearly decaying importance for exploration as a function of percent of area explored.
    - The total area is approximated by the user-inputted geofence area.

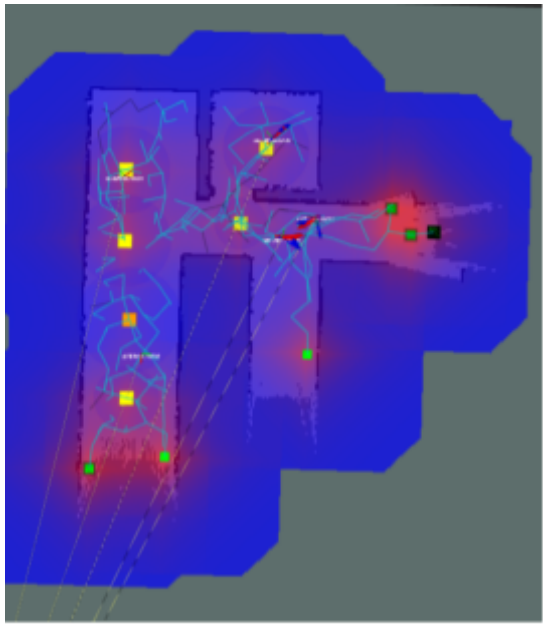


Fig 19. Information gain visualization of each task. Red is higher, blue is lower.

The final HIGH objective we want to maximize is then

objective = (2 * utility – dist_cost) * info_gain * e2_weights

After calculating the highest priority tasks to assign next, we then optimally assign the tasks to each robot based on distance cost using the Hungarian algorithm. The total distance needed to be traveled for all robots is then minimized. The performance of HIGH compared to Naive can be seen in figure 20.

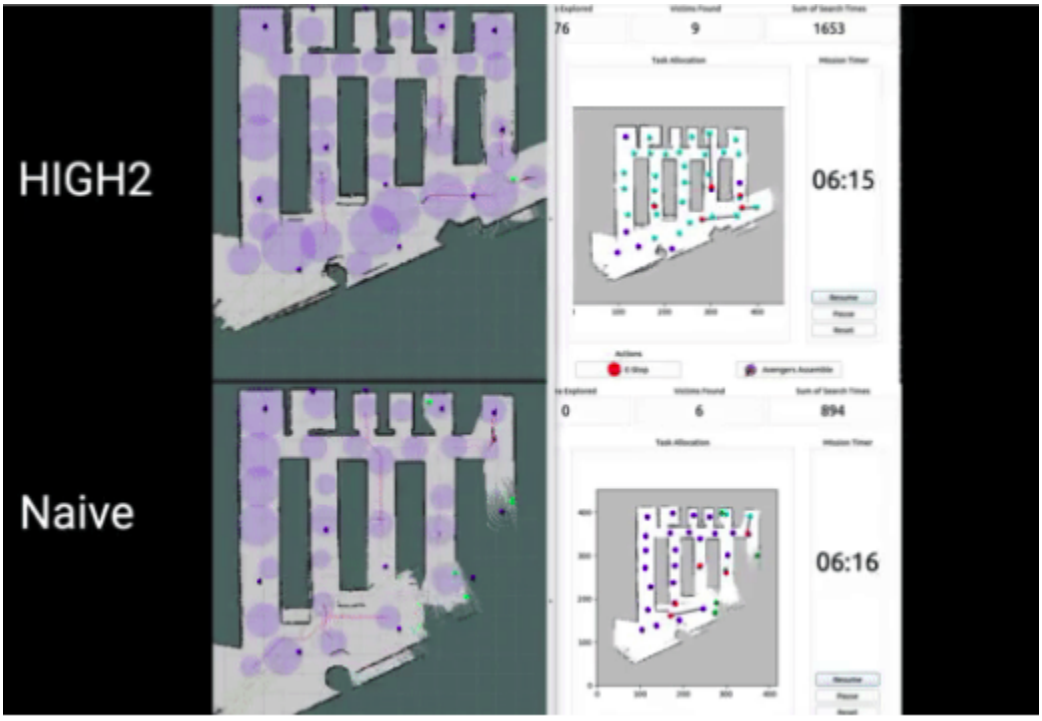


Fig 20. HIGH2 vs Naive task allocation. You can see that HIGH both explores more of the environment and finds more victims than Naive in the same amount of time.

Additionally, the task allocation subsystem is also able to dynamically reallocate tasks. For example, if an agent runs out of battery or crashes during search, the system is able to take the unfinished tasks and reallocate them to the remaining agents such that all tasks are still completed.

8.1.7 GUI

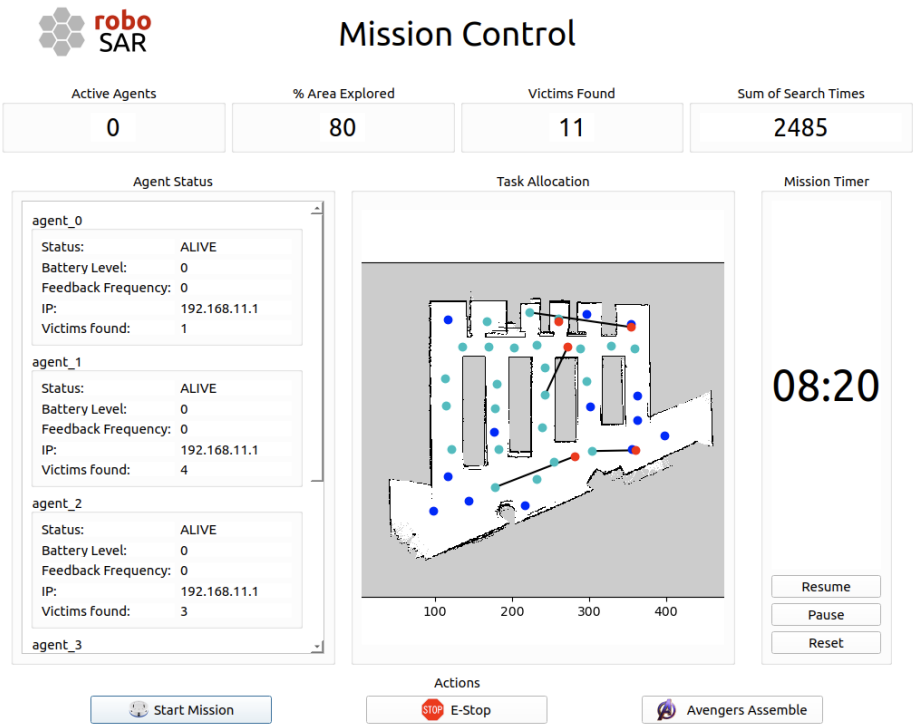


Fig 21. Mission Control GUI

The Mission Control GUI (figure 21) is a user interface where the user running the mission can monitor and interact with the search and rescue system. It is built with PyQT5 and directly interfaces with ROS to easily be integrated with the rest of the system.

8.1.8 Simulation

The RoboSAR system also includes a multi-robot simulator which is based on the ROS Stage simulator (figure 22) that we can use for testing and debugging our algorithms before transferring them to real hardware. This saved us a lot of time in both spring and fall semesters.

We also set up our system to be compatible with the Gazebo simulator for Apriltag detection and reporting testing.

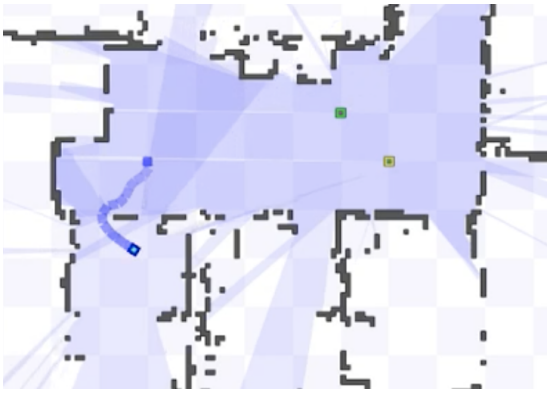


Fig 22. ROS Stage Simulator with multiple robots.

8.2 Modeling, Analysis, and Testing

Table 5. Subsystem Tests

Test #	Objective	Process	Requirements
1	To ensure that generated map and location of victims are accurate	1. Place AprilTags (victims) in environment 2. Determine ground-truth positions of victims 3. Set up world origin AprilTag 4. Set up agents at starting position so they can see AprilTag 5. Teleoperate agents to explore environment and locate victims	Generated map is accurate and stable (D.F.0) Found victims are reported and localized (M.F.0) Position estimates of victims are accurate (M.P.1)
2	Ensure robots are exploring an unknown environment in minimal time.	1. Set up agents in the test area. 2. Ensure frontiers are accurately generated for the said area. 3. Observe that agents are being assigned to the correct frontiers that minimize cost and maximize utility. 4. Agents stop when there are no more frontiers to expand.	System shall map and search in unknown environment simultaneously during operation (D.F.0)
3	To ensure that the path planned avoids collision with obstacles known in the map and other agents	1. Set up the static environment with obstacles. 2. Start the agent bringup software, along with the autonomy stack. 3. Send different goal points to multiple robots with paths in collision. 4. Observe the path being followed by the robots to go from its source point to the goal point. 5. Repeat this by giving multiple goal points. 6. Observe any collisions.	The robot does not collide with any obstacle or agent while traversing to the goal point.
4	To ensure that generated map and location of agents are accurate	1. In simulation, set up agents at starting position 2. Teleoperate agents to explore environment	Generated map is accurate and stable (D.F.0) Position estimates of agents are within 30 cm of true position after loop closures
5	To ensure precise victim detection for a multi-robot fleet	1. Place AprilTags at 9 known locations on the testing area 2. Start Mission Control and give tasks to the 3 agents such that all the AprilTags can be detected by the agents . 3. Once an AprilTag is detected as a ‘delta-pose’ value from a particular agent, compute the pose of the AprilTag with respect to world frame and compare it with the above ground truth. 4. Repeat the experiment until all AprilTags are spotted.	The reported victim locations are accurate within 5m. The communication is lossless with ignorable latency.

8.3 FVD Performance Evaluation

Most of the mandatory requirements of the system were met during SVD. The remaining mandatory requirements as shown in Table 6: M.F.0, M.P.0 and M.P.1 were demonstrated and validated successfully during the FVD. There were a total of 12 victims spread out in the test

environment and the final life score was the sum of the time taken to detect each victim with a penalty added for all the victims not found. In Demo 1 of FVD, we ran the naive task allocator which was able to find only 8 victims with the life score of 4537. This did not satisfy our requirement of finding atleast 75% of the victims in the environment. In Demo 2, we ran the improved HIGH task allocator, which was able to find 10 victims with a life score of 3459. With this we validated M.P.0, and also proved that HIGH performed better with the life score being 23% better than that of naive. The map of the environment was not available to the system and we had heterogeneous tasks - frontier and coverage, to map and search the environment simultaneously. With this, we were also able to validate the desirable requirements mentioned in Table 7.

For FVD Encore, we demonstrated the addition of a robot to the fleet during operation and proved that our system can handle changes in the fleet size. The performance of the system was great during FVD but was significantly improved for FVD encore where we were able to detect 100% of the victims in the given time of 8 minutes. We were able to satisfy all of our desirable requirements shown in table 7.

Table 6. FVD Mandatory requirements

M.F.0.	System shall identify and localize victims in the environment using fiducial markers within desired rescue time.
M.P.0.	System will detect 75% of victims in the environment.
M.P.1.	System will localize victim positions within 5m.
M.N.2.	System shall have a user interface to add/subtract robots during operation.

Table 7. FVD Desirable requirements

D.F.0.	System shall map and search in an unknown environment simultaneously during operation.
D.P.0.	System will finish searching the environment 20% faster than the naive algorithm in an unknown environment.
D.N.2.	System shall support heterogenous robot search and control.

8.4 Strong and Weak Points

8.4.1 Strengths

- SLAM is robust to dynamic obstacles, changing maps, and bad laser scans.
- Fleet management system is scalable and provides a good platform for future development.
- Task generation is very robust and works well even for changing maps.
- HIGH task allocation greatly outperforms Naive.
- The system rarely has any collisions thanks to a well-designed controller.
- System is able to handle changing fleet sizes and still perform well.

- Whole system is map agnostic!
- We have created reusable open-source multi-robot software.

8.4.2 Weaknesses

- Limited mobility capabilities of the Khepera robots limits testing environments.
- Path planning can sometimes have a long compute time. Possible improvements include downsampling of the map and making computations run in parallel.
- Robots can not travel in unknown space. This sometimes leads to unnecessarily long paths. Instead, we could also plan in unknown space but with a bigger cost than free space. This also means we will have to keep recomputing paths at a certain frequency during execution.
- Scalability of the fleet is limited by communication: with more than 5 agents, we observed wireless communication throttling. Solutions to this would include making the system more decentralized, meaning moving more computations from the server to individual agents.
- Large delay in camera image capture.
- Manual focusing of cameras is required.
- Task allocation has a limited planning horizon.
- No local planning.

9. Project management

9.1 Schedule

Overall, we were able to stick pretty closely to our initial project schedule (figure 21). We successfully broke each subsystem down into parallelizable tasks, which allowed each team member to work without blockers. Another thing we did well was being flexible in manpower, meaning that if a subsystem was lagging behind in progress, we were able to efficiently reassign people to help push that subsystem forward and catch up to the rest of the system's development. One failure was that because of time constraints, we descope swarm control. However, this turned out to be not much of a loss because we were operating in relatively open environments which did not necessarily need a lot of swarm formations for navigation.

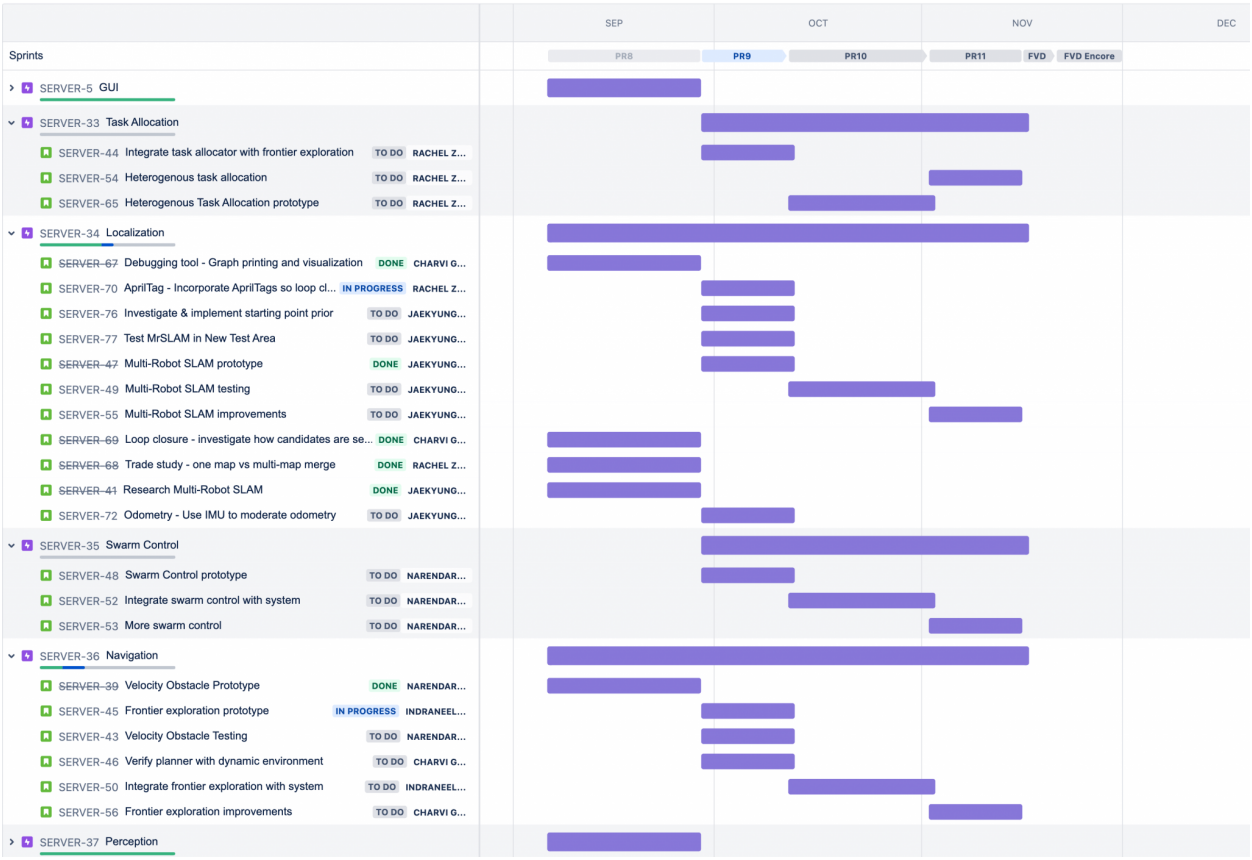


Fig 21. Fall 2022 Schedule

9.2 Budget

The initial budget allotted to RoboSAR was $5000. The cost for this project is low. The project is largely software based, so very few physical components are needed. The Khepera robots are provided by our sponsor, so they do not add to the cost. Additionally, some of the sensors we are interested in are available in the MRSD inventory, so they will not add to the cost either. However, because we have many agents, most items must be ordered in large quantities.

9.2.1 Parts List

Table 8. Parts list

Name	Description	Source	ROM Cost ($)	Qty	Subtotal ($)
Khepera IV	Robot platform for agents	Sponsor	0	10	0
YDLIDAR X4	LIDAR	COTS	0	8	0
Wifi Router	For robot-server communication	Amazon	349.99	1	349.99
Wifi Repeater	For robot-server communication	Amazon	29.99	3	89.97

PCB parts	For debug PCB	Digikey	78.29	1	78.29
Duckiebot	Summer experimentation	Duckietown	294.99	1	294.99
Containers	Features for SLAM	Amazon	171.41	Many	171.41
Power Supplies	Power supply for router, server	Amazon	300.00	4	300.00
Team Merchandise	Stickers, Hoodies	Same day custom	501.94	2	501.94
Gadgets for productivity	Mouse, Keyboard	Amazon	344.08	Many	344.08
Event logistics	TV	CMU Facilities	600	2	600
Miscellaneous	Snacks	Amazon	1400	1	1400
Total					4130.68

9.2.2 Budgeting Process

The Budgeting process was carefully thought out and items were evaluated to see if it was within the budget. However, since we did not have any hardware big ticket items as the platform was already provided with and the sensors were obtained from the inventory, our parts only involved items to facilitate smooth functioning of our systems such as gadgets and containers used as features. At each time frame, we evaluated if adding a certain item to the budget list would result in an overshoot

9.3 Risk Management

Table 9. Fall 22 Risks

No.	Risk	Likelihood	Consequence	Mitigation Plan
1	Network connectivity problems in the testing area	3	4	Explore WiFi repeaters
2	MultiRobot SLAM is computationally heavy	3	5	Purchase better compute
3	New sensors (IMU and camera) leading to battery drain	4	3	Test to analyze effects / employ battery packs
4	Latency of camera	4	2	Synchronize data using timestamps

	streaming/apriltag detection causes incorrect victim localization			
5	Limited FOV and range of camera causes longer search times	3	3	Process high resolution camera frames using parallel processing
6	Cannot find suitable test environment	4	4	Construct alternative environments



Fig 22. Likelihood-Consequence of Fall 22 risks

We executed the mitigation plans by using wifi repeaters for 1, using good CPUs for 2, installing battery packs for 3, synchronizing LRF data using timestamps for 4, using IPC for 5, using Tepper hallways for 6.

10. Conclusions

We are very proud that RoboSAR not only delivered the mandatory requirements, but also two moonshot desired requirements of unknown environment S&R and heterogeneous task allocation. This needed quite an overhaul of our individual subsystems in the fall compared to the spring, but our high level architectural blocks (of centralized decision making, centralized localisation and centralized navigation) proved to be very extendable to the new scope. We also significantly improved the system itself compared to the spring semester with more robots, faster robots and also a much smarter task allocation policy which we are very happy about! Here are some lessons we learnt along the way.

10.1 Lessons Learned

Choice of robots: The project started off with Khepera robots are the choice of hardware for our system and due to a multitude of reasons they proved to be not quite right for the vision of the project to do search & rescue in natural environments. Due to lack of ROS support we spent a lot of time implementing and debugging basic communication which could have been avoided. Actuated wheels next to rigid castor wheels meant that they are only suitable for table top operation which severely limited our testing environments. We found the sensors on the robot to also not be quite robust for long term operation.

Shortening the testing loop with simulation: Something that helped our scheduling a lot throughout the project was the availability of our multi-robot simulator. For instance, our localization system was ready for testing, but the hardware was not. Instead, we used our simulator to continue development and subsystem integration. The simulator was able to unblock a lot of subsystem co-dependencies and also allowed us to develop features in parallel that would otherwise need to be serial.

Strong emphasis on good quality code and version control: Since the start we employed strategies like

- Version control for github and no merges without code reviews
- Documentation of code design and test-driven development
- Using coding standards, coding styles and good coding practices

All of these practices helped speed up our development tremendously and reduced debugging time.

Agile integration cycles with demo focused PRs: Our subsystems were never developed in isolation right from the start. We used the simulator as a first line of integration testing, and used Hackathon Days and Progress Reviews as deadlines for full system integration demos. We had a system oriented software development approach which really helped ensure subsystems are being developed in close harmony with each other.

Hardware focused architecture development: A good understanding of the hardware you are working with is key for feasible systems architectures. As we gained familiarity with our Khepera robotic platforms, we realized that a lot of our initially planned functions for the robots were infeasible because of low compute and we had to offload these to the server side.

10.2 Future Work

We are currently working on at least two **open source contributions** and two **research papers** based on our work in RoboSAR. For this some activities which are ongoing are: code cleanup, creating pull requests into base repositories (like slam_toolbox), and working with repository maintainers on code reviews. We are also researching baselines for our contributions in SLAM, task generation and task allocation to prove the novelty and performance of our methods.

Investigating **scalability of the fleet management system** is something that we could not get to in time. We could only test the FMS successfully with 5-6 robots and it will be interesting to find

out the reason for sketchy communication beyond this number, is it the limitation of the router or the limitations of the robot hardware itself?

We based our new task allocation on the assumption that we have no information on the humans in the environment and there is no way to sample data from this information but in reality the scenario is not that bleak. Something we would like to extend the task allocation to is being able to **incrementally estimate the distribution of humans** in the environment and then use approaches like ergodic search to balance exploration and exploitation.

MrSLAM required a lot of tuning of loop closure parameters for robust performance, which makes the current approach less generalizable. Because environmental features can vary greatly, it becomes intractable to involve manual tuning for every new environment. An interesting direction of work could be to use learning based techniques to **make MrSLAM more generalizable** to different environments.

An interesting direction of work which we could not get to is **manipulating obstacles** in the environment to aid search & rescue. Our system does not interact with the environment at all, but in all realistic scenarios, the environment can be very cluttered and robots have to plow through obstacles while searching for humans. Collaborative manipulation of larger obstacles could also get very interesting.

11. References

1. W. Burgard, M. Moors, C. Stachniss and F. E. Schneider, "Coordinated multi-robot exploration," in IEEE Transactions on Robotics, vol. 21, no. 3, pp. 376-386, June 2005, doi: 10.1109/TRO.2004.839232.
2. Simmons, R. and Apfelbaum, D. and Burgard, W. and Fox, D. an Moors, M. and Thrun, S. and Younes, H. "Coordination for Multi-Robot Exploration and Mapping", Proceedings of the AAAI National Conference on Artificial Intelligence, 2000.
3. B. Yamauchi, "A frontier-based approach for autonomous exploration," Proceedings 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation CIRA'97. 'Towards New Computational Principles for Robotics and Automation', 1997, pp. 146-151, doi: 10.1109/CIRA.1997.613851.
4. H. Umari and S. Mukhopadhyay, "Autonomous robotic exploration based on multiple rapidly-exploring randomized trees," 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2017, pp. 1396-1402, doi: 10.1109/IROS.2017.8202319.
5. M. Kulkarni, M. Dharmadhikari, M. Tranzatto, S. Zimmermann, V. Reijgwart, P. De Petris, H. Nguyen, N. Khedekar, C. Papachristos, L. Ott, R. Siegwart, M. Hutter, and K. Alexis, "Autonomous Teamed Exploration of Subterranean Environments using Legged and Aerial Robots", 2022.
6. L. M. Miller, Y. Silverman, M. A. MacIver and T. D. Murphey, "Ergodic Exploration of Distributed Information," in IEEE Transactions on Robotics, vol. 32, no. 1, pp. 36-52, Feb. 2016, doi: 10.1109/TRO.2015.2500441.
7. Macenski, S., Jambrecic I., "SLAM Toolbox: SLAM for the dynamic world", Journal of Open Source Software, 6(61), 2783, 2021.
8. J. van den Berg, Ming Lin and D. Manocha, "Reciprocal Velocity Obstacles for real-time multi-agent navigation," 2008 IEEE International Conference on Robotics and Automation, 2008, pp. 1928-1935, doi: 10.1109/ROBOT.2008.4543489.
9. D. Wilkie, J. van den Berg and D. Manocha, "Generalized velocity obstacles," 2009 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2009, pp. 5573-5578, doi: 10.1109/IROS.2009.5354175.
10. Korsah GA, Stentz A, Dias MB. "A comprehensive taxonomy for multi-robot task allocation". The International Journal of Robotics Research. 2013;32(12):1495-1512. doi:10.1177/0278364913496484
11. J. P. Queralta et al., "Collaborative Multi-Robot Search and Rescue: Planning, Coordination, Perception, and Active Vision," in IEEE Access, vol. 8, pp. 191617-191643, 2020, doi: 10.1109/ACCESS.2020.3030190.